Lab 15 – Implementation of Storage Allocation Strategy (Stack Allocation)

Aim

To implement stack storage allocation using push and pop operations.

Algorithm

1. Start the program.
2. Initialize stack pointer top = -1.
3. Read the number of elements.
4. Insert elements into the stack using **push**.
5. During push:
    1. If stack is full, report **overflow**.
    2. Otherwise increment top and insert element.
6. Remove elements using **pop**.
7. During pop:

    1. If stack is empty, report **underflow**.
    2. Otherwise remove the top element and decrement top.

8. Display stack contents.
9. Stop.

C Program

```
#include <stdio.h>

#define MAX 10

int stack[MAX];
int top = -1;

void push(int value) {
   if (top == MAX - 1) {
      printf("Stack Overflow\n");
      return;
   }

   stack[++top] = value;
}
void pop() {
   if (top == -1) {
      printf("Stack Underflow\n");
      return;
   }
   printf("Popped element: %d\n", stack[top--]);
}
```

```
void display() {
    if (top == -1) {
        printf("Stack is empty\n");
        return;
    }

    printf("Stack elements: ");
    for (int i = top; i >= 0; i--)
        printf("%d ", stack[i]);
    printf("\n");
}

int main() {
    int n, value;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter element %d: ", i + 1);
        scanf("%d", &value);
        push(value);
    }

    display();

    pop();
    display();

    return 0;
}
```

Sample Input

Enter number of elements: 3
Enter element 1: 10
Enter element 2: 20
Enter element 3: 30

Sample Output

Stack elements: 30 20 10
Popped element: 30
Stack elements: 20 10

Result

Thus the stack storage allocation strategy was implemented successfully.